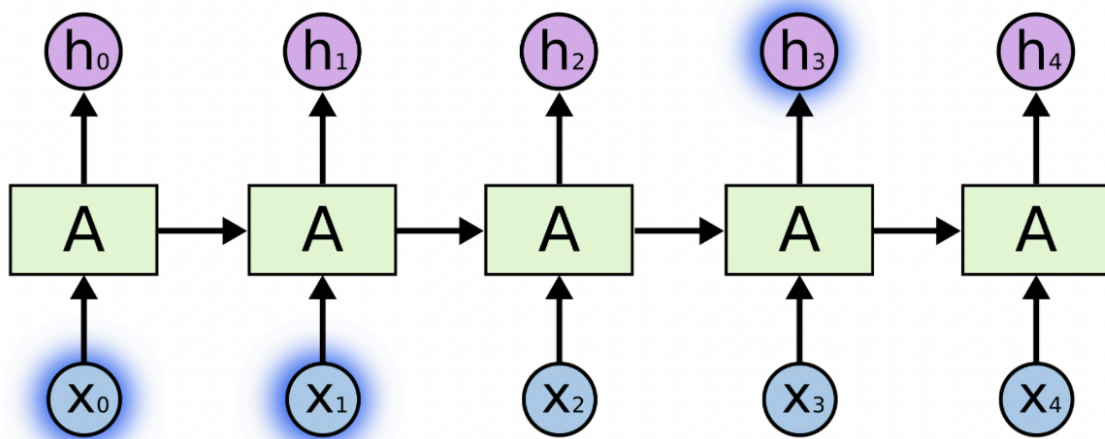


Why RNN is a problem for long-term dependencies

Short-term dependency (RNN works fine)

Example: “The colour of the sky is _____”

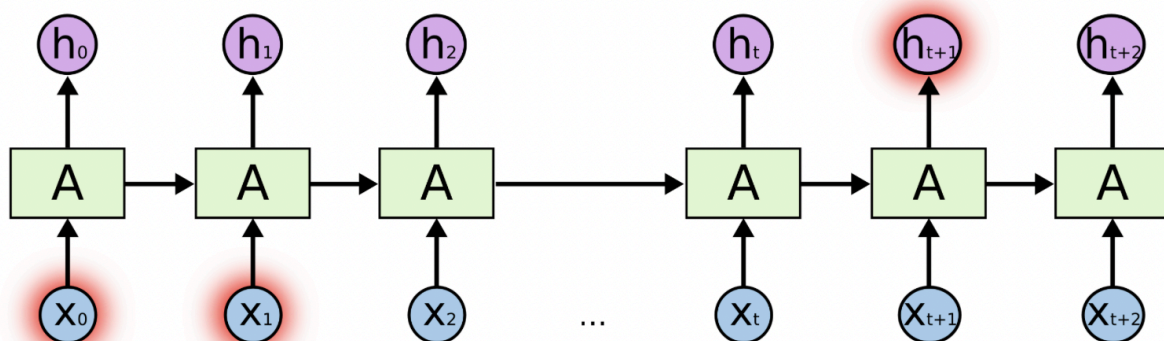
Here, to predict the missing word “blue”, the useful context “sky” appears just 1-2 steps back. Since the dependency is so short, the gradient flowing backwards doesn’t vanish and a vanilla RNN can easily learn this mapping.



Long-term dependency (RNN struggles)

Example: “I grew up in France. I love food, and I speak fluent _____”.

To correctly predict *French*, the model must remember the word “France”, which appeared several timesteps earlier. When we backpropagate through time over all those steps, the gradient becomes very small (due to repeated multiplication), so the influence of “France” basically disappears by the time it reaches the weights. Hence, vanilla RNNs fail to capture long-range context.



How gradients flow backwards through time in an RNN

During training, RNNs use **Backpropagation Through Time (BPTT)**. The idea is to unroll the RNN over all timesteps and pass the gradient backwards one step at a time using the chain rule.

We care about how the loss depends on the recurrent weight matrix $\mathbf{W}_h$, because that determines how the model should update itself.

$$\frac{\partial L}{\partial W_h} = \frac{\partial L}{\partial h_T} \cdot \frac{\partial h_T}{\partial h_{T-1}} \cdot \frac{\partial h_{T-1}}{\partial h_{T-2}} \cdots \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_h}$$

The goal is to compute the gradient of the loss with respect to W_h , which determines how we should update the weights during training.

The first term $\frac{\partial L}{\partial h_T}$, represents how sensitive the loss is to the final hidden state at

time-step T . The next term $\frac{\partial h_T}{\partial h_{T-1}}$, captures how the final hidden state depends on the

hidden state from the previous step, $T-1$. Each subsequent term, such as $\frac{\partial h_{T-1}}{\partial h_{T-2}}$ through to

$\frac{\partial h_2}{\partial h_1}$, reflects how each hidden state depends on the one before it. Finally, $\frac{\partial h_1}{\partial W_h}$ tells us how the very first hidden state is affected by the recurrent weight matrix. Multiplying all these terms together gives the overall influence of the recurrent weights on the final loss. Since this is a long chain of multiplications, the gradient can end up either shrinking (vanishing) or growing (exploding), which is the core problem in training vanilla RNNs.

Each term like

$$\frac{\partial h_T}{\partial h_{T-1}}$$

contains:

- the recurrent weight W_h

- and the derivative of the activation function (e.g., tanh, sigmoid)

$$\frac{\partial h_t}{\partial h_{t-1}} \approx W_h \cdot f'(a_t)$$

Since, at each time step(t),

$$h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

So, using chain rule,

$$\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial f(a_t)}{\partial a_t} \cdot \frac{\partial a_t}{\partial h_{t-1}}$$

And that results in,

$$\frac{\partial h_t}{\partial h_{t-1}} = f'(a_t) \cdot W_h$$

(since, because the only part of a_t that depends on h_{t-1} is the term $W_h h_{t-1}$)

And because $\tanh'$ or $\text{sigmoid}' \in (0,1)$, multiplying these terms repeatedly over T steps leads to either:

Vanishing Example

$T = 10, W_h = 0.9, f' \approx 0.5$

Gradient factor = $(0.9 \times 0.5)^{10} \approx 0.00035 \Rightarrow$ almost zero $\Rightarrow$ early context forgotten.

Exploding Example

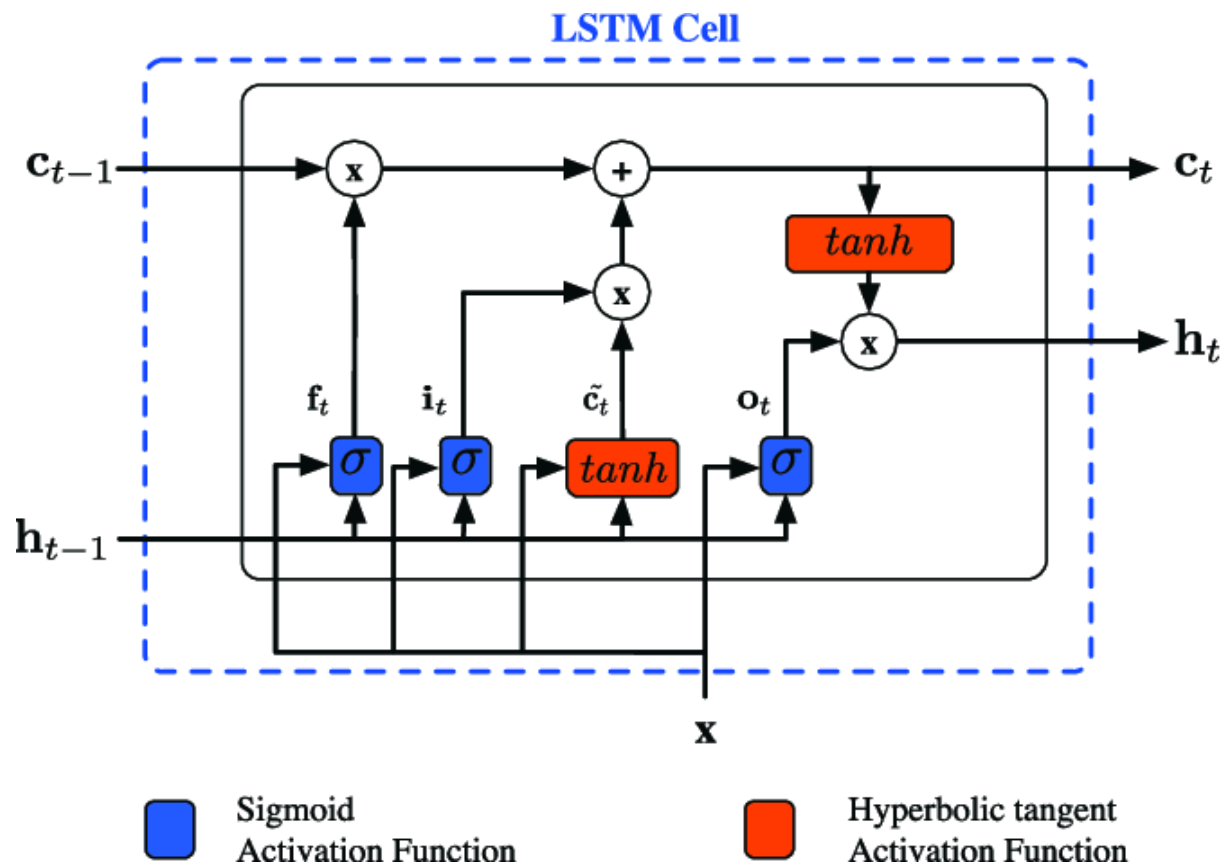
$T = 10, W_h = 1.2, f' \approx 0.9$

Gradient factor = $(1.2 \times 0.9)^{10} \approx 2.14 \Rightarrow$ very large $\Rightarrow$ unstable.

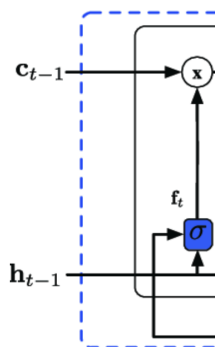
So:

- **Vanishing gradient** $\Rightarrow$ model stops learning long-term context
- **Exploding gradient** $\Rightarrow$ weight updates become unstable and cause divergence

LSTM (Long Short-Term Memory)



FORGET GATE



FORGET GATE

The forget gate is the first blue sigmoid block (σ) in an LSTM and produces the vector f_t . Its only job is to decide **how much of the previous cell state (c_{t-1})** should be retained.

- **Inputs:**
 - Current input vector x_t
 - Previous hidden state h_{t-1}

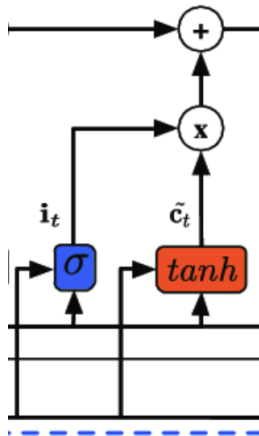
- **Computation:**

$$z_f = W_f x_t + U_f h_{t-1} + b_f$$

$$f_t = \sigma(z_f) \rightarrow \text{gives values} \in (0,1), \text{ acting like soft switches.}$$
- **Apply to old memory:**

$$c_t^1 = f_t \odot c_{t-1}$$
- **Example:**
 If $f_t = [0.755, 0.377, 0.713]$ and $c_{t-1} = [1.0, 2.0, 3.0] \Rightarrow c_t^1 = [0.755, 0.754, 2.139]$
 Only **75.5%, 37.7%, 71.3%** of the old memory survives.
- **Intuition:**
 If c_{t-1} stored topic, timeline, character details, the forget gate checks the next input (x_t) and decides which are still relevant, removing outdated pieces so the network isn't overwhelmed as it processes long sequences.

INPUT GATE AND CANDIDATE GATE



Once the forget gate has pruned old memory, the LSTM decides **what new information to write** in.

- **Input gate:**

$$z_i = W_i x_t + U_i h_{t-1} + b_i$$

$$i_t = \sigma(z_i) \rightarrow \text{percentage of new memory to write (vector of values} \in (0,1))$$
- **Candidate (potential memory):**

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \rightarrow \text{values} \in (-1,1)$$
- **Write to memory:**

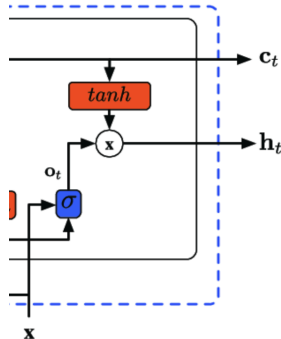
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
- **Example:**

$$i_t = [0.62, 0.21, 0.90], \quad \tilde{c}_t = [0.8, -0.5, 0.25]$$

$$\rightarrow \text{new contribution} = [0.496, -0.105, 0.225]$$
 This gets added to the partially forgotten old state c_{t-1} .
- **Intuition:**
 Candidate contains **what** could be written and the input gate decides **how much** to actually write, so only crucial new context is stored.

Candidate $\rightarrow$ $\times$ Input Gate $\rightarrow$ Long-Term Memory Update

OUTPUT GATE



After updating the cell state c_t , we need to decide **what part to expose** as the hidden state h_t , which gets passed to the next timestep.

- **Gate calculation:**

$$z_o = W_o x_t + U_o h_{t-1} + b_o$$

$$o_t = \sigma(z_o)$$

- **Generate potential short-term output:**

$$\tilde{h}_t = \tanh(c_t)$$

- **Actual hidden state:**

$$h_t = o_t \odot \tilde{h}_t$$

- **Example:**

$$c_t = [1.251, 0.649, 2.364], o_t = [0.77, 0.44, 0.21]$$

$$\rightarrow \tilde{h}_t = \tanh(c_t) \approx [0.848, 0.571, 0.982]$$

$$\rightarrow h_t \approx [0.653, 0.251, 0.206]$$

- **Intuition:**

Output gate determines how much of the updated memory should be revealed to the next word, and how much should remain hidden internally.

EXAMPLE

Sentence: "John studied very hard for months, and finally passed the exam."

- **Forget gate** trims less-important memory (e.g. studying timeline) but keeps critical parts (John + exam context).
- **Input gate + candidate** encode "passed" $\rightarrow$ "success" and decide how strongly to write it.
- **Output gate** releases a positive hidden signal indicating success, which is then passed to process the next word ("the").

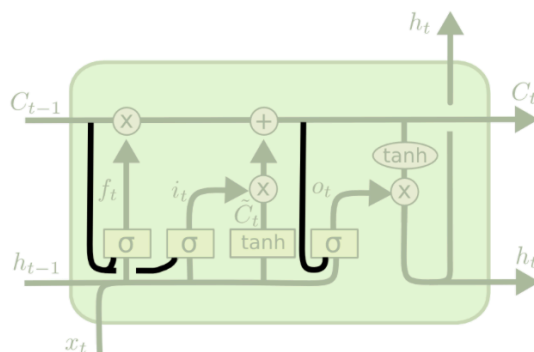
Summary:

Forget $\rightarrow$ decides what to keep,

Input+Candidate -> decides what to add,
Output -> decides what to show.

LSTM Variants

1. Peephole Variant



$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

In a *standard* LSTM, all three gates (forget, input, output) make their decisions based only on:

- current input x_t
- previous hidden state h_{t-1}

In a **peephole LSTM**, each gate is also given access to the previous cell state c_{t-1} (or sometimes the current c_t), allowing it to “peek” directly into the long-term memory itself.

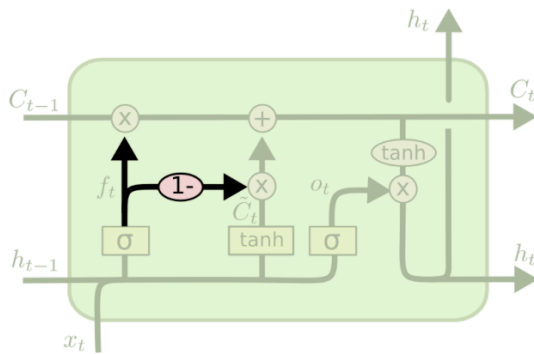
Why is this useful?

Sometimes the memory cell contains timing/context information which can help the gates make better decisions.

With peephole connections:

- Forget gate checks c_{t-1} to judge if something is worth keeping.
- Input gate checks if memory is already full/empty before writing new content.
- Output gate peeks at c_t before deciding how much to expose.

2. Coupling Forget Gate and Input Gate Variant



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

In a regular LSTM:

- Forget gate (f_t) controls how much of C_{t-1} we keep
- Input gate (i_t) controls how much of $\tilde{C}_t$ (new candidate) we write

This variant assumes:

$$i_t = 1 - f_t$$

So we only learn one gate (usually the forget gate), and use its complement as the input gate.

- If $f_t \approx 1 \rightarrow$ keep most old memory, write very little new
- If $f_t \approx 0 \rightarrow$ forget old memory, write lots of new

This acts like a dial balancing “remembering” vs. “updating”, and also reduces parameters.

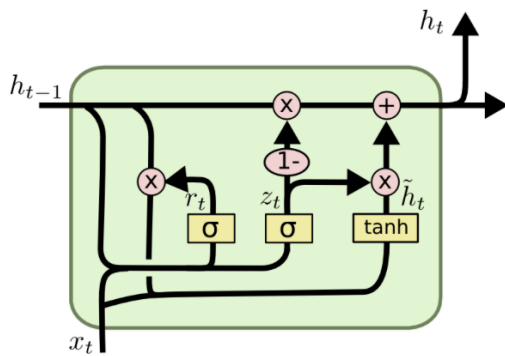
3. GRU Variant

LSTMs solved the vanishing gradient problem, but came with some drawbacks:

Limitations of LSTM:

1. Too many gates $\rightarrow$ heavy architecture
2. Large number of parameters $\rightarrow$ higher memory + risk of overfitting
3. Slower inference $\rightarrow$ computationally expensive

GRU simplifies the design by merging gates and removing the separate cell state.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

1) Update Gate -> z_t

$$z_t = \text{sigmoid}(W_z \cdot [h_{t-1}, x_t])$$

Controls how much of the previous hidden state (h_{t-1}) is carried over.

- z_t close to 1 -> update a lot (let new information override old)
- z_t close to 0 -> update very little (keep most of the old hidden state)

(Acts like a combination of forget + input gate in LSTM.)

2) Reset Gate -> r_t

$$r_t = \text{sigmoid}(W_r \cdot [h_{t-1}, x_t])$$

Controls how much of the previous hidden state should be used to form the new memory.

- $r_t \sim 0$ -> forget the past
- $r_t \sim 1$ -> use the past heavily

3) Candidate Hidden State -> $\tilde{h}_t$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

New information proposal, formed by mixing the reset hidden state with current input.

4) Final Hidden State -> h_t

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

So instead of fully replacing the old hidden state h_{t-1} , GRU blends old and new based on z_t :

- If $z_t \approx 1$ -> $h_t \approx 0 * h_{t-1} + 1 * \tilde{h}_t$ -> use mostly new candidate memory ($\tilde{h}_t$)
- If $z_t \approx 0$ -> $h_t \approx 1 * h_{t-1} + 0 * \tilde{h}_t$ -> keep old hidden state (h_{t-1})

This h_t is then passed to the next timestep